```
Start coding or generate with AI.
```

Assignment Supervised machine Learning - Regression

House Price Prediction Using Bolton Housing Data

This dataset provides information about house prices in Bolton. House Price Prediction

In this notebook, you'll follow the basic machine learning process to build a regression model to predict house prices using the "Boston Housing Dataset" from sklearn. The regression model will either be a Decision Tree or Random Forest regressor.

Follow the instructions and complete each TODO to complete the assessment on the essential steps in building and evaluating a regression model.

The following is a description of each column in the dataset:

Dataset Features (Bolton Housing):

- CRIM: Crime rate by town
- ZN: Proportion of residential land zoned for large lots
- INDUS: Proportion of non-retail business acres per town
- CHAS: Charles River dummy variable (1 if tract bounds river; 0 otherwise)
- NOX: Nitric oxide concentration (parts per 10 million)
- RM: Average number of rooms per dwelling
- AGE: Proportion of owner-occupied units built before 1940
- DIS: Weighted distances to five Boston employment centers
- RAD: Index of accessibility to radial highways
- TAX: Full-value property tax rate per `$10,000`
- PTRATIO: Pupil-teacher ratio by town
- B: Proportion of Black population
- LSTAT: Percentage of lower status of the population
- MEDV (Target): Median value of owner-occupied homes in `$1,000s`

Dataset is from sklearn Datasets

```
```

```
# --- Imports ---
# TODO: Import all the necessary libraries for data handling, visualization, ar
# Example: import pandas as pd
# Add your imports here:
# --- Imports ---
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score


# --- Data Collection and Loading ---
# TODO: Load the 'Boston Housing' dataset from sklearn and convert it into a pa
# Hint: Use `load_boston()` from `sklearn.datasets`

# Load dataset and convert to DataFrame:

# Add your code here:
# --- Load the Boston Housing Dataset from OpenML ---
boston = fetch_openml(name="boston", version=1, as_frame=True)
df = boston.frame.copy()
df.rename(columns={'medv': 'MEDV'}, inplace=True)
df['CHAS'] = df['CHAS'].astype('category')

# Quick look
df.head()


# --- Quick Check of Data ---
# TODO: Display the first few rows of the dataset to understand its structure.
# Hint: Use `.head()` to inspect the first few rows.

# Add your code here:

# --- Quick Check ---
df.info()
df.describe()
df.isnull().sum()

# TODO: Check the features and target variable. Identify which is continuous ar
# Hint: Use `.info()` and `.describe()` to inspect data types and statistical p

# Add your code here:
```

```
# --- EDA and Data Preprocessing ---
# TODO: Check for missing/null values.
# Hint: Use `.isnull().sum()` to check for null values.

# Add your code here:


# TODO: Visualize the data. Create scatter plots to see the relationship betwee
# Example: Use `plt.scatter()` to visualize the relationship between features l

# Add your code here:

# --- Correlation Heatmap ---
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

# --- Automated scatter function ---
def plot_feature_vs_target(df, target_col="MEDV"):
    for col in df.columns.drop(target_col):
        plt.figure(figsize=(6,4))
        plt.scatter(df[col], df[target_col], alpha=0.6)
        plt.xlabel(col); plt.ylabel(target_col)
        plt.title(f"{col} vs {target_col}")
        plt.show()

# plot_feature_vs_target(df, "MEDV")   # optional

# TODO: Create a function to automate scatter plots for all features vs the tar
# Hint: The function should loop over a list of features and plot scatter plots

# Define your function here:


# TODO: Use the function to visualize the relationships between multiple featur
# Example: ['RM', 'LSTAT', 'AGE', 'CRIM']
# Target: 'MEDV'

# Add your code here:


# --- ML Model Training ---
# TODO: Split the dataset into training and testing sets.
# Hint: Use `train_test_split()` from `sklearn.model_selection` with an 80/20 s

# Define X (features) and y (target) and perform the train-test split:
# --- Split the dataset ---
X = df.drop(columns=["MEDV"])
```

```
X = pd.get_dummies(X, drop_first=True)   # encode CHAS if needed
y = df["MEDV"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# --- Choose model (Random Forest) ---
rf = RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train)


# TODO: Choose an appropriate regression model: Decision Tree or Random Forest.
# Hint: Use either `DecisionTreeRegressor` or `RandomForestRegressor` from `skl

# Define your regression model here:


# TODO: Train the model on the training data.
# Hint: Use `.fit()` to train the model.

# Add your code here:
# --- Evaluate ---
pred = rf.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, pred))
r2 = r2_score(y_test, pred)

print(f"Base Random Forest Model -> RMSE: {rmse:.2f}")
print(f"R² Score: {r2:.3f}")


# --- Model Evaluation ---
# TODO: Evaluate the performance of the model on the test set using relevant me
# Hint: Use `mean_squared_error()` and `r2_score()` from `sklearn.metrics`.

# Perform parameter tuning on the model if needed to improve the performance of

# Add your parameter tuning code here:

# Predict on the test set and calculate the evaluation metrics:
# --- Grid Search for better performance ---
param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [None, 10, 20],
    "min_samples_split": [2, 5]
}

grid = GridSearchCV(RandomForestRegressor(random_state=42),
                    param_grid, cv=3, scoring="r2", n_jobs=-1)
grid.fit(X_train, y_train)

print("Best parameters:", grid.best_params_)
```

```
best = grid.best_estimator_
pred = best.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, pred))
r2 = r2_score(y_test, pred)
print(f"Tuned Model -> RMSE: {rmse:.2f}, R²: {r2:.3f}")


# --- Model Prediction ---
# TODO: Predict house prices from a new set of feature inputs.
# Example new data: Use hypothetical or randomly generated values for the featu
#
# Example new data: CRIM = 0.2, ZN = 12.5, INDUS = 7.07, CHAS = 0, NOX = 0.5, R

# Add your prediction code here:

# --- Predict on new sample ---
sample = {
 "CRIM":0.1,"ZN":18.0,"INDUS":2.31,"CHAS":0,"NOX":0.538,"RM":6.575,"AGE":65.2,
 "DIS":4.09,"RAD":1.0,"TAX":296.0,"PTRATIO":15.3,"B":396.9,"LSTAT":4.98
}
x_new = pd.DataFrame([sample])
x_new = pd.get_dummies(x_new, drop_first=True).reindex(columns=X.columns, fill_
predicted_price = best.predict(x_new)[0]
print(f"Predicted House Price (MEDV): {predicted_price:.2f} (in $1000s)")
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   CRIM     506 non-null    float64
 1   ZN       506 non-null    float64
 2   INDUS    506 non-null    float64
 3   CHAS     506 non-null    category
 4   NOX      506 non-null    float64
 5   RM       506 non-null    float64
 6   AGE      506 non-null    float64
 7   DIS      506 non-null    float64
 8   RAD      506 non-null    category
 9   TAX      506 non-null    float64
 10  PTRATIO  506 non-null    float64
 11  B        506 non-null    float64
 12  LSTAT    506 non-null    float64
 13  MEDV     506 non-null    float64
dtypes: category(2), float64(12)
memory usage: 49.0 KB
```

